

Documentación Técnica del Proyecto Handwritten Digit Recognition

Fabio Alonso

January 17, 2026

Contents

1	Introducción	2
2	Estructura del Proyecto	2
3	Descripción de Archivos	2
3.1	load_mnist.py	2
3.2	model.py	2
3.3	train.py	3
3.4	predict.py	3
3.5	gui.py	3
4	Flujo General del Sistema	4
5	Conclusión	4

1 Introducción

Este proyecto implementa un sistema de reconocimiento de dígitos manuscritos basado en redes neuronales convolucionales (**CNN**), entrenadas sobre el dataset **MNIST**. El sistema incluye entrenamiento, predicción y una interfaz gráfica que permite al usuario dibujar dígitos y proporcionar retroalimentación (*feedback*) para mejorar el modelo.

2 Estructura del Proyecto

El proyecto está organizado en los siguientes archivos Python:

- `gui.py`
- `load_mnist.py`
- `model.py`
- `predict.py`
- `train.py`

Cada archivo tiene una función específica dentro del flujo completo del sistema, descrita a continuación.

3 Descripción de Archivos

3.1 `load_mnist.py`

Función principal: Cargar y preprocesar el dataset MNIST para su uso en entrenamiento o prueba.

- Descarga automática del dataset si no existe.
- Transformaciones aplicadas a las imágenes:
 - Conversión a tensor (`ToTensor()`).
 - Normalización de píxeles entre $[-1,1]$ usando `Normalize((0.5,), (0.5,))`.
- Crea un `DataLoader` para iterar sobre los datos en mini-batches.
- Permite visualizar imágenes de ejemplo para verificar los datos.

3.2 `model.py`

Función principal: Definir la arquitectura de la red neuronal (**DigitCNN**) para clasificar dígitos manuscritos.

- Dos capas convolucionales con activación ReLU y *MaxPooling*.
- Dos capas totalmente conectadas (fully connected) para clasificación.
- Método `forward` que define cómo fluyen los datos a través de la red.

3.3 train.py

Función principal: Entrenar la red neuronal usando MNIST.

- Inicializa modelo en GPU o CPU.
- Define función de pérdida (`CrossEntropyLoss`) y optimizador (`Adam`).
- Itera sobre múltiples épocas:
 - Forward pass para obtener predicciones.
 - Backward pass para calcular gradientes y actualizar pesos.
 - Cálculo de métricas (*accuracy*) por época.
- Guarda el modelo entrenado en `models/digit_cnn.pth`.

3.4 predict.py

Función principal: Realizar inferencia sobre nuevas imágenes usando el modelo entrenado.

- Carga el modelo previamente entrenado.
- Aplica las mismas transformaciones de preprocesamiento.
- Desactiva gradientes para optimizar predicción.
- Devuelve la clase más probable.

3.5 gui.py

Función principal: Proporcionar una interfaz gráfica interactiva para dibujar dígitos, predecirlos y permitir retroalimentación para mejorar el modelo.

- Canvas de dibujo para que el usuario dibuje un dígito.
- Preprocesamiento automático:
 - Centrado y escalado de la imagen.
 - Inversión de colores si es necesario.
 - Resize a 28×28 .
- Predicción utilizando el modelo y muestra de probabilidades en histograma.
- Corrección del usuario (*feedback*):
 - Permite indicar la clase correcta si la predicción fue errónea.
 - Guarda la imagen en un directorio específico para reentrenamiento rápido.
- Fine-tuning del modelo usando un subconjunto de MNIST y los datos de feedback.
- Botones: *Predict*, *Clear*, *Update Model*, y botones de corrección (0-9).

4 Flujo General del Sistema

1. Preparación de datos (`load_mnist.py`).
2. Definición del modelo (`model.py`).
3. Entrenamiento del modelo (`train.py`).
4. Uso del modelo para predicción (`predict.py`).
5. Interacción y retroalimentación del usuario (`gui.py`).

5 Conclusión

El sistema implementa un flujo completo de aprendizaje supervisado, inferencia y mejora continua mediante feedback del usuario. La estructura modular asegura mantenibilidad, escalabilidad y reutilización del código, permitiendo la integración futura de nuevas arquitecturas de red, datasets o funcionalidades adicionales.